VIOLATION REGISTER
SMART TRAFFIC VIOLATION LOGGER (FLASK)
PROJECT REPORT

Done By: Nandana Pramod

Department: Artificial Intelligence and Data Science

Institution: Rajalakshmi Engineering College

Introduction

The Violation Register (Smart Traffic Violation Logger) is a web application developed using Python and Flask. It has two sides. On the **officer side**, traffic police log in, record the vehicle number, violation type, location, date and fine amount for each violation, search a vehicle's history, filter records and mark fines as Paid. On the **citizen side**, a vehicle owner logs in and can only view the challans issued to their own vehicle(s) and pay them.

Every record is stored in an SQLite database through SQLAlchemy and is marked Unpaid automatically. Each record also has a digital challan with a dynamically generated QR code. Scanning the QR code opens a public status page that shows the violation details and the payment status, with no login required.

The interface is built with Bootstrap and is responsive. Officer pages are protected by a secure login, citizens are blocked from every officer page, and the first run loads sample data so the system can be demonstrated immediately. The project demonstrates Flask routing, role-based access control, database integration, QR code generation and basic CRUD operations.

Prerequisites

- Programming – Python (functions, dictionaries, decorators)
- Web Framework – Flask (routes, Jinja2 templates, sessions) with HTML, CSS and Bootstrap for the front end
- Database basics – tables, primary keys and the idea of an ORM (SQLAlchemy)
- Required Python Libraries: Flask, Flask-SQLAlchemy, Flask-Login, qrcode and Pillow
- Basic understanding of HTTP routes (GET/POST), form handling, redirects and user roles

Technologies Used

Python: the primary programming language for the whole backend.

Flask: builds the web server, defines the routes and renders HTML pages through Jinja2 templates.

SQLAlchemy and SQLite: Flask-SQLAlchemy maps Python classes to tables. SQLite keeps all data in one file (instance/violations.db), so no database server is needed.

Flask-Login and Werkzeug security: manage login sessions and store passwords as salted hashes. A role field separates officers from citizens.

qrcode and Pillow: generate the QR code image for each challan.

HTML / CSS / Bootstrap 5: build the responsive interface, including the navigation bar, forms, tables and the printable challan.

Install Required Libraries

```
pip install flask
pip install flask-sqlalchemy
pip install flask-login
pip install "qrcode[pil]"

# or everything at once:
pip install -r requirements.txt
```

Python Libraries Used

- **Flask:** routing, sessions, flash messages and templates.
- **Flask-SQLAlchemy:** the User and Violation models and every database query.
- **Flask-Login:** login, logout and route protection (login_required, current_user).
- **werkzeug.security:** generate_password_hash and check_password_hash.
- **qrcode (with Pillow):** creates the QR code image in memory.
- **Standard library:** re, io, socket, datetime, functools, os, threading and webbrowser.

Coding — Step by Step Instructions

Step 1: Create the Project Structure

```
traffic_logger/
├── app.py              (models, routes, QR generation, sample data)
├── requirements.txt
├── templates/          base, login, register, form, history, challan,
│                       public_status, status_lookup, account,
│                       my_challans, pay
└── static/             style.css and local Bootstrap + fonts
```

Step 2: Import Modules and Configure the App

```
import io, os, re, socket, threading, webbrowser
from datetime import date, datetime, timedelta
from functools import wraps
import qrcode
from flask import (Flask, abort, flash, redirect, render_template,
                   request, send_file, url_for)
from flask_login import (LoginManager, UserMixin, current_user,
                         login_required, login_user, logout_user)
from flask_sqlalchemy import SQLAlchemy
from werkzeug.security import (check_password_hash,
                               generate_password_hash)

app = Flask(__name__)
app.config["SECRET_KEY"] = os.environ.get("SECRET_KEY", "dev-secret")
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///violations.db"
db = SQLAlchemy(app)
login_manager = LoginManager(app)
login_manager.login_view = "login"
```

Step 3: Define the Database Models

```
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True, nullable=False)
    password_hash = db.Column(db.String(200), nullable=False)
    role = db.Column(db.String(10), default="officer")  # or "citizen"
    vehicles = db.Column(db.String(200), default="")    # citizen's plates

class Violation(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    vehicle_number = db.Column(db.String(20), nullable=False, index=True)
    violation_type = db.Column(db.String(50), nullable=False)
    location = db.Column(db.String(120), nullable=False)
    date = db.Column(db.Date, nullable=False)
    fine_amount = db.Column(db.Float, nullable=False)
    status = db.Column(db.String(10), default="Unpaid")
```

The User table holds both kinds of account; the role column says whether the user is an officer or a citizen, and a citizen's vehicles column lists the plates they own. The id of a Violation row doubles as the challan number.

Violation column	Type	Description
id	Integer (primary key)	Challan number, generated automatically
vehicle_number	String (indexed)	Registration number, upper case, no spaces
violation_type	String	No Helmet, Overspeeding, Signal Jumping, etc.
location / date	String / Date	Where and when the violation happened
fine_amount	Float	Fine in rupees
status	String	Unpaid (default) or Paid

Step 4: Helper Functions

```
def clean_vehicle(value):
    return re.sub(r"[\s\-]", "", value or "").upper()
```

clean_vehicle makes "tn09 ab-1234" and "TN09AB1234" identical so that searching is reliable. A second helper, read_form, validates the add / edit form and returns an error message instead of crashing on bad input.

Step 5: Role-Based Access and Login

```
def role_required(role, fallback):
    def decorator(view):
        @wraps(view)
        def wrapped(*args, **kwargs):
            if current_user.role != role:
                flash(f"That page is for {role}s only.", "warning")
                return redirect(url_for(fallback))
            return view(*args, **kwargs)
        return login_required(wrapped)
    return decorator

officer_required = role_required("officer", "my_challans")
citizen_required = role_required("citizen", "history")
```

```
@app.route("/login", methods=["GET", "POST"])
def login():
    role = request.values.get("role", "officer")   # Officer / Citizen tab
    if request.method == "POST":
        user = User.query.filter_by(
            username=request.form["username"].strip()).first()
        if user and check_password_hash(user.password_hash,
                                        request.form["password"]):
            if user.role != role:
                flash("Use the " + user.role.title() + " tab.", "warning")
            else:
                login_user(user)
                return redirect(url_for("home"))   # home picks the side
        else:
            flash("Wrong username or password.", "danger")
    return render_template("login.html", role=role)
```

The login page has an Officer tab and a Citizen tab. The password is compared with the stored hash, and an account can only sign in through the tab that matches its role. Every officer route carries @officer_required and every citizen route @citizen_required, so each side is locked away from the other.

Step 6: Add a Violation (Officer)

```
@app.route("/add", methods=["GET", "POST"])
@officer_required
def add():
    if request.method == "POST":
        data, error = read_form(request.form)
        if not error:
            v = Violation(**data, status="Unpaid")   # always Unpaid
            db.session.add(v)
            db.session.commit()
            return redirect(url_for("challan", vid=v.id))
        flash(error, "danger")
    return render_template("form.html", f=request.form, editing=False)
```

Step 7: View History with Search and Filters (Officer)

```
@app.route("/history")
@officer_required
def history():
    q = Violation.query
    vehicle = clean_vehicle(request.args.get("vehicle"))
    if vehicle:
        q = q.filter(Violation.vehicle_number.contains(vehicle))
    if request.args.get("type"):
        q = q.filter_by(violation_type=request.args["type"])
    if request.args.get("status"):
        q = q.filter_by(status=request.args["status"])
    # date_from / date_to add >= and <= conditions on Violation.date
    records = q.order_by(Violation.date.desc()).all()
    unpaid = [r for r in records if r.status == "Unpaid"]
    ...   # summary: count, unpaid count, outstanding fines
```

Each filter that the officer fills in narrows the query, and the filters can be combined. Clicking a number plate in the table opens that vehicle’s complete history.

Step 8: Update Status, Edit and Delete (Officer)

```
@app.route("/violation/<int:vid>/pay", methods=["POST"])
@officer_required
def mark_paid(vid):
    v = db.get_or_404(Violation, vid)
    v.status = "Paid"
    db.session.commit()
    return redirect(request.referrer or url_for("history"))
```

The Mark paid button changes the status from Unpaid to Paid. The edit route reuses the add form to correct a record, and the delete route removes one after a confirmation prompt. These routes accept only POST, so opening a link cannot change data.

Step 9: Generate the QR Code and the Public Status Page

```
def public_url(vid):
    host = request.host
    if host.split(":")[0] in ("127.0.0.1", "localhost"):
        host = lan_ip() + ":" + host.partition(":")[2]   # phone-friendly
    return f"{request.scheme}://{host}" + url_for("public_status", vid=vid)

@app.route("/qr/<int:vid>.png")
def qr(vid):
    v = db.get_or_404(Violation, vid)
    img = qrcode.make(public_url(v.id), box_size=8, border=1)
    buf = io.BytesIO()
    img.save(buf, format="PNG"); buf.seek(0)
    return send_file(buf, mimetype="image/png")

@app.route("/status/<int:vid>")             # public, no login
def public_status(vid):
    v = db.get_or_404(Violation, vid)
    return render_template("public_status.html", v=v)
```

The QR image is created on the fly and is never stored. It encodes the address of the public status page. When the app is opened as localhost, the computer's network address is used instead, because a phone cannot open 127.0.0.1 (lan_ip finds that address with the socket module).

Step 10: The See Status Page

Visitors without the QR code can open See Status and enter the challan number and the vehicle number. The route looks the challan up and redirects to its public status page only when both values match, which stops people from browsing other vehicles' records.

Step 11: Citizen Registration, My Challans and Payment

```
def citizen_plates():
    return [p for p in current_user.vehicles.split(",") if p]

@app.route("/my-challans")
@citizen_required
def my_challans():
    plates = citizen_plates()
    records = Violation.query.filter(
        Violation.vehicle_number.in_(plates)).order_by(
        Violation.status.desc(), Violation.date.desc()).all()  # unpaid first
    ...   # summary: total, unpaid, amount due

@app.route("/pay/<int:vid>", methods=["GET", "POST"])
@citizen_required
def pay(vid):
    v = db.get_or_404(Violation, vid)
    if v.vehicle_number not in citizen_plates():
        abort(404)                    # not this citizen's vehicle
    if request.method == "POST" and request.form.get("method"):
        v.status = "Paid"             # demo payment: no money moves
        db.session.commit()
        return redirect(url_for("my_challans"))
    return render_template("pay.html", v=v)
```

A citizen can create an account at /register by giving a username, a password and one or more vehicle numbers. The My Challans page shows only challans whose vehicle number is in the citizen's list, unpaid first, together with the total amount due. Pay now opens a payment page where a method (UPI, card or net banking) is chosen. The payment is simulated: confirming changes the status to Paid. A citizen cannot open or pay a challan that belongs to another vehicle.

Step 12: Officer Account Page

The /account route has two forms: change the logged-in officer's password (after checking the current one) and add another officer account. Both store only password hashes, and the page is available to officers only.

Step 13: Create the Database, Load Sample Data and Run

```
with app.app_context():
    db.create_all()
    upgrade_old_database()         # adds role / vehicles to old files
    if not User.query.first():     # first run only
        load_demo_data()           # 1 officer, 2 citizens, 16 challans

if __name__ == "__main__":
    if not os.environ.get("WERKZEUG_RUN_MAIN"):
        url = "http://127.0.0.1:5000"
        threading.Timer(1.2, lambda: webbrowser.open(url)).start()
    app.run(host="0.0.0.0", port=5000, debug=False,
            use_reloader=True)
```

On the first run the tables are created and demo data is added: one officer, two citizens and sixteen challans for seven vehicles with a mix of paid and unpaid fines. The dates are calculated from today so the data always looks current. The browser opens automatically, and host 0.0.0.0 lets a phone on the same Wi-Fi reach the app.

Step 14: Bootstrap Front End

base.html holds the navigation bar, which changes with the user: officers see Add Violation, View / Search History, See Status, Account and Log out; citizens see only My Challans and Log out; visitors see See Status with Officer Login and Citizen Login. Every other template extends it. challan.html

shows the printable challan with the QR code and a PAID / UNPAID stamp, my_challans.html and pay.html form the citizen side, and Bootstrap and the fonts are stored locally so the app works offline.

Output

The application was run locally with the sample data that loads on the first run. The following accounts were used for testing:

Role (login tab)	Username	Password	Access
Officer	officer	police123	All 16 challans; add, edit, delete, search
Citizen	ravi	ravi123	Own challans: TN09AB1234, TN22EF9090
Citizen	meena	meena123	Own challans: TN10CD5678

The project was opened in VS Code and started from the terminal with python app.py.

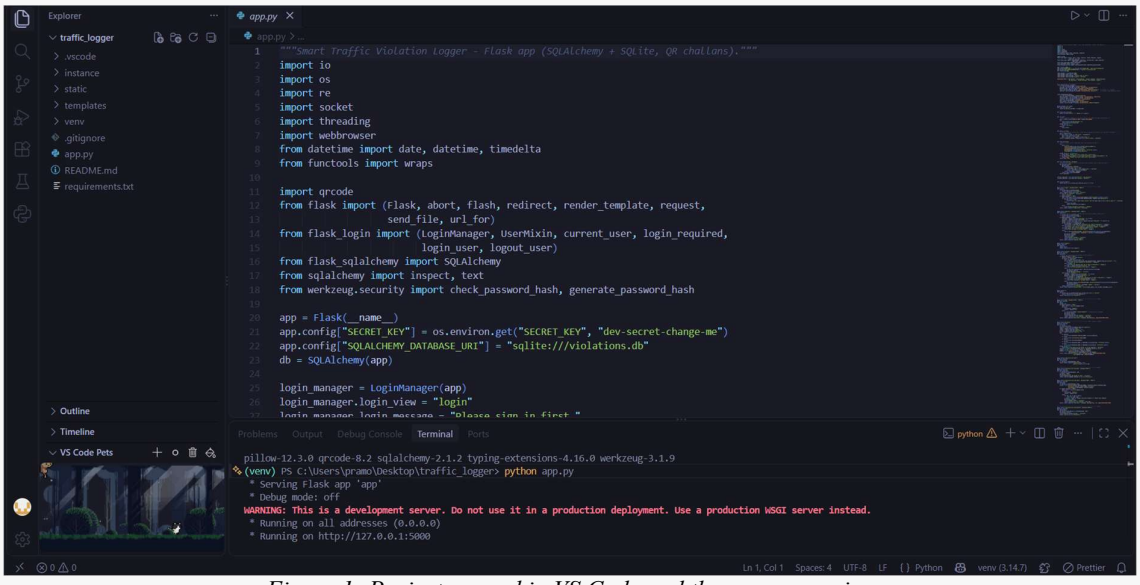


Figure 1: Project opened in VS Code and the server running

The login page has an Officer tab and a Citizen tab. Only a logged-in officer can add or modify records.

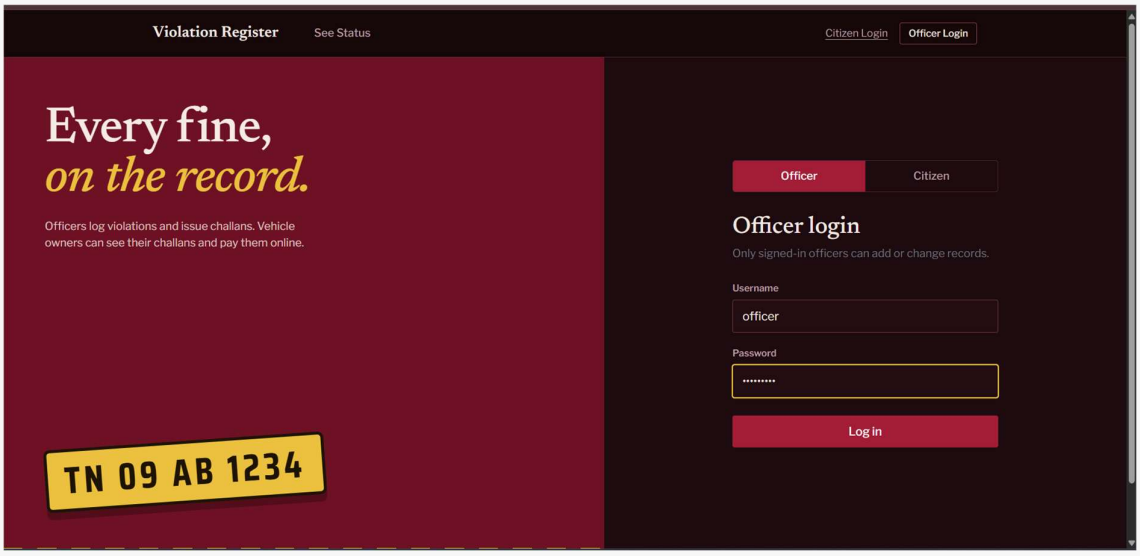


Figure 2: Officer login

The officer enters the vehicle number, violation type, location, date and fine amount. The record is saved as Unpaid.

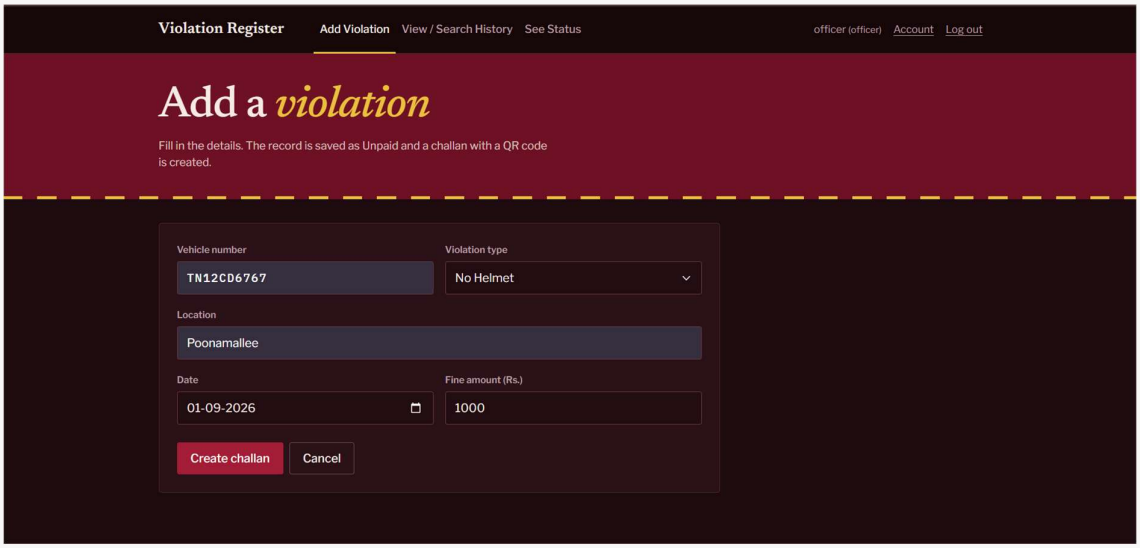


Figure 3: Add Violation form

After saving, the digital challan is generated with a QR code and an UNPAID stamp.

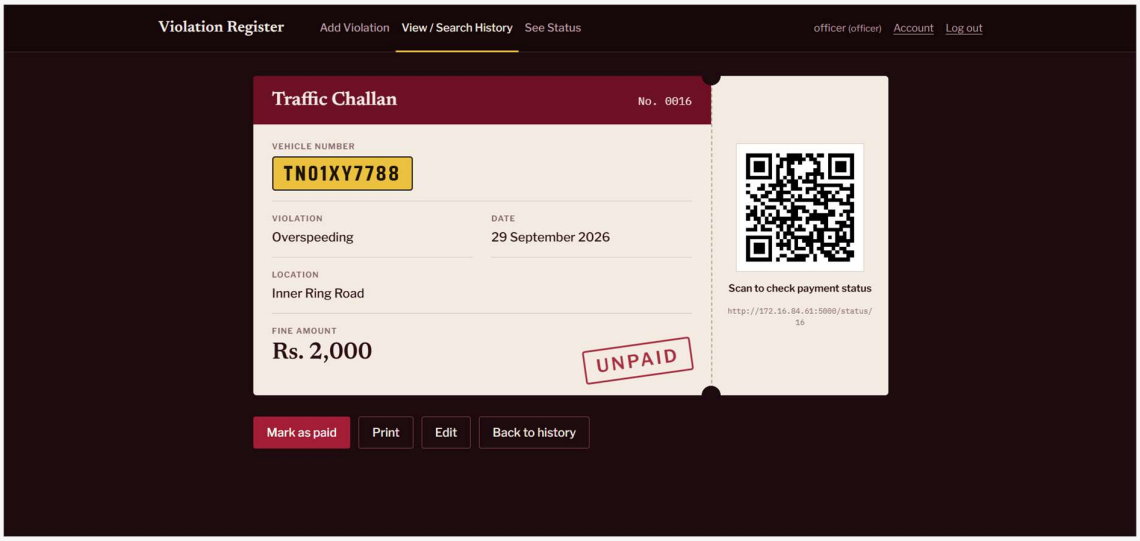


Figure 4: Challan with QR code

Scanning the QR code opens a public page with the violation details and payment status, with no login.

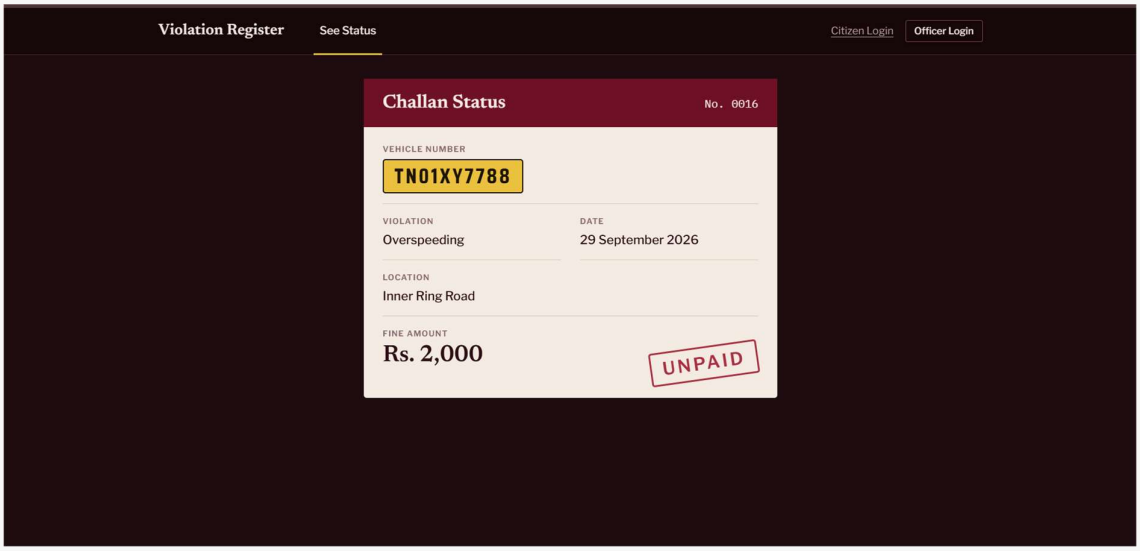


Figure 5: Public status page opened from the QR code

The history page lists every violation with the number of records, unpaid records and outstanding fines.

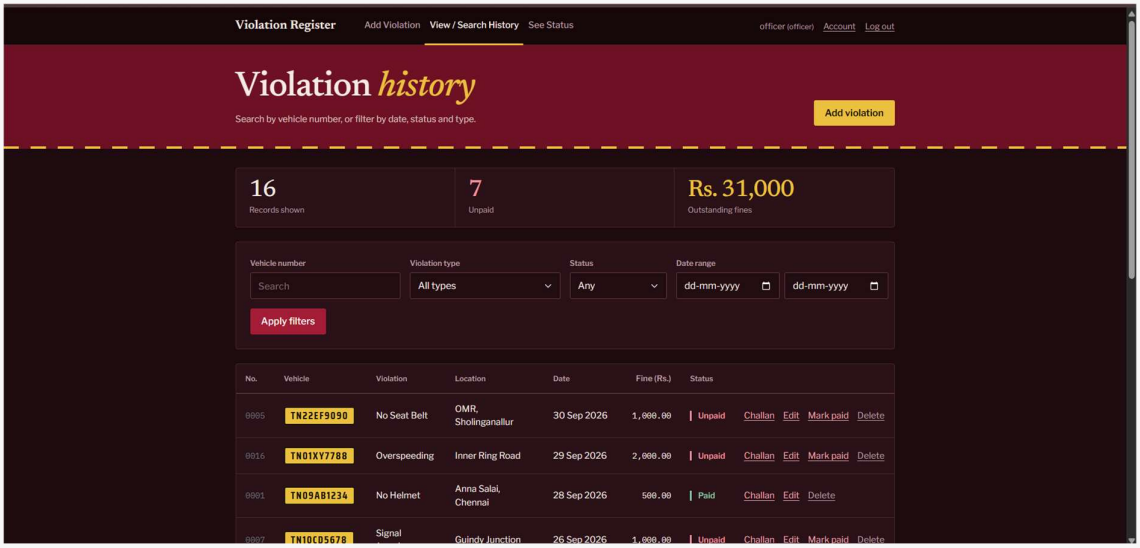


Figure 6: Violation history with sample data

Searching a vehicle number, or clicking a plate, shows that vehicle's history. Status, type and date filters can be combined.

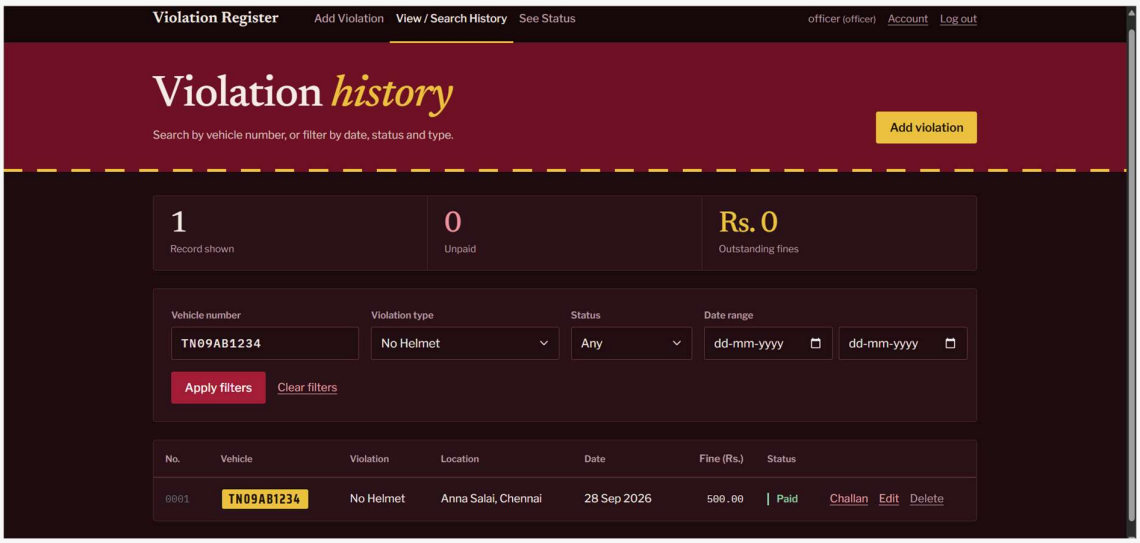


Figure 7: Search and filters

Anyone can check a fine without logging in by entering the challan number and vehicle number.

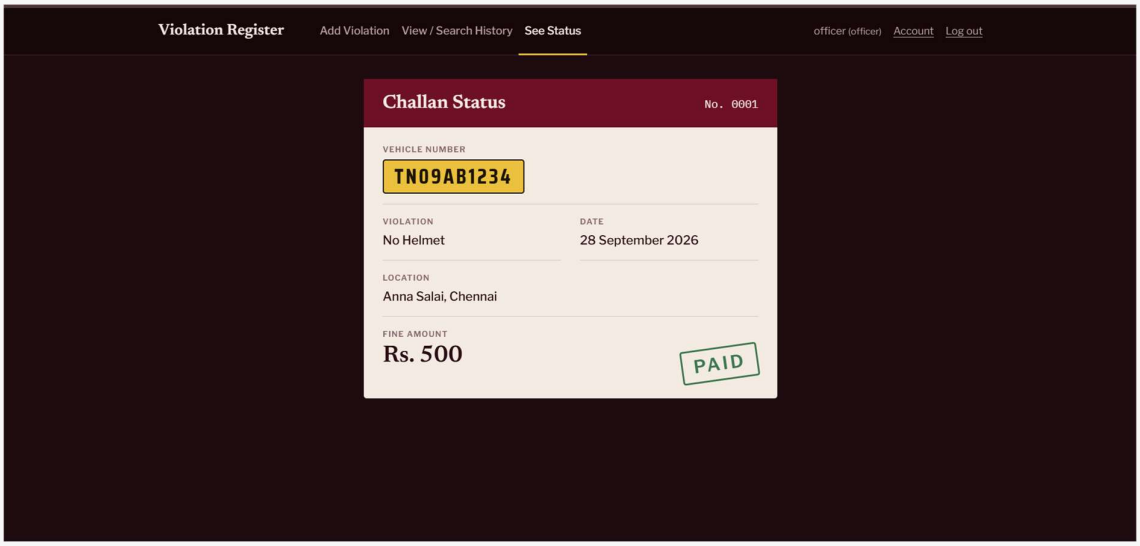


Figure 8: See Status page

After logging in, a citizen sees only the challans of their own vehicles, with the amount due.

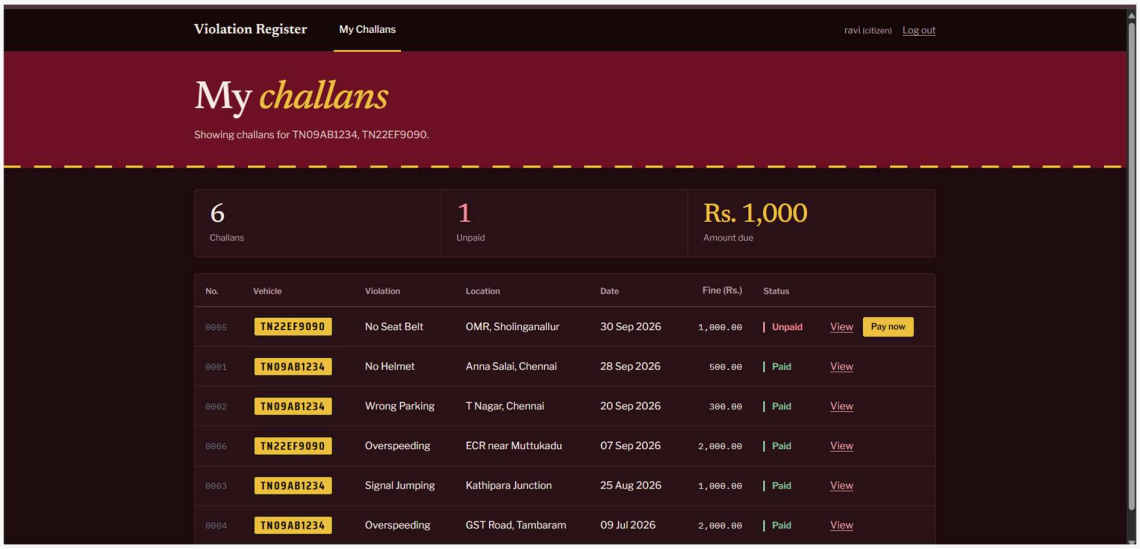


Figure 9: My Challans (citizen side)

Pay now opens the payment page, where the method is chosen and the payment is confirmed.

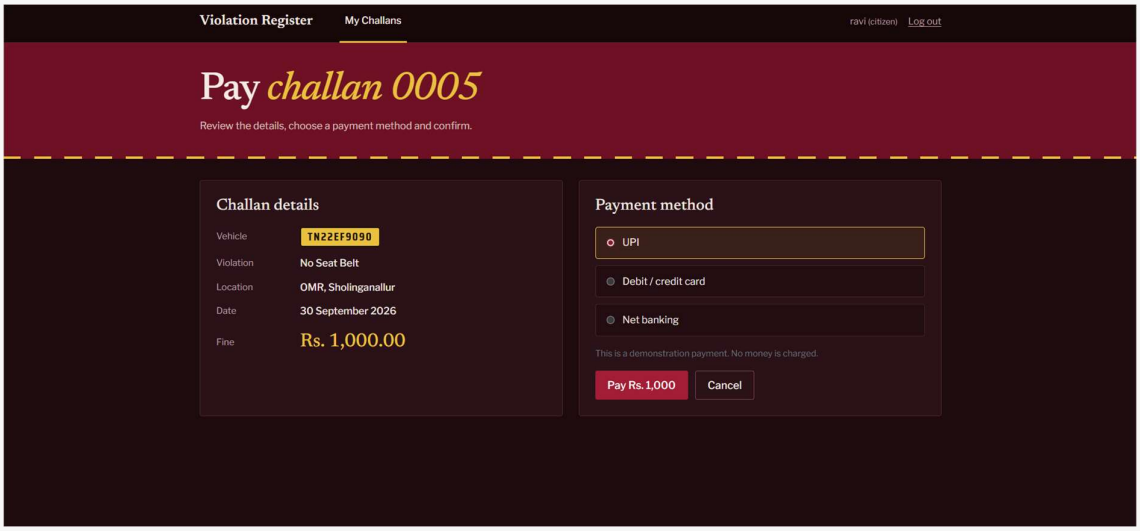


Figure 10: Payment page

All data is stored in the user and violation tables of the SQLite database.

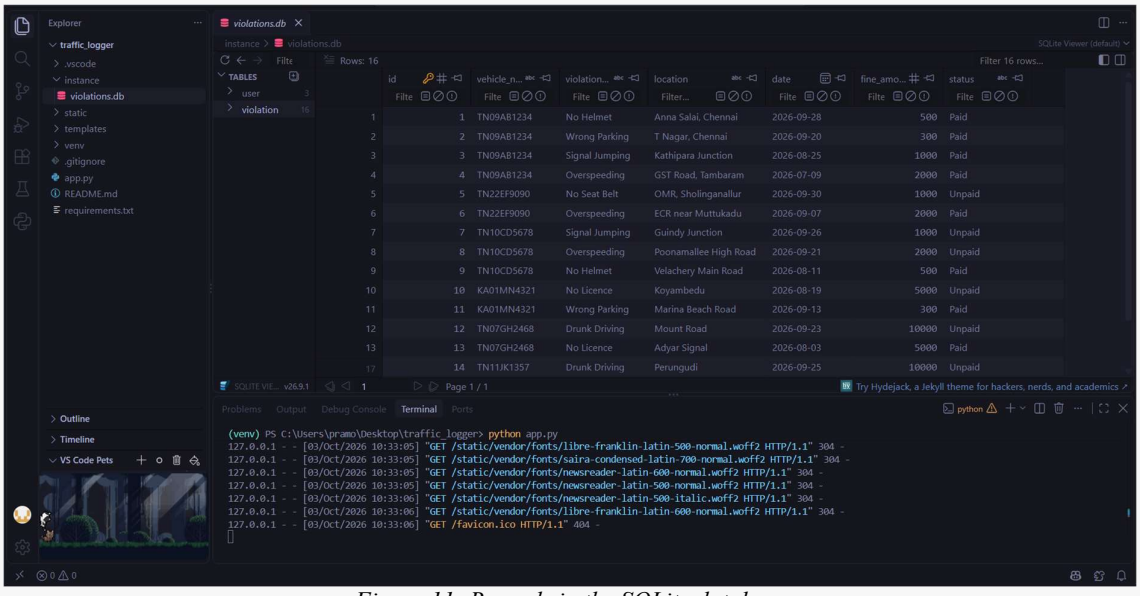


Figure 11: Records in the SQLite database

Troubleshooting Table

Issue	Cause	Solution
ModuleNotFoundError: No module named 'flask' / 'qrcode'	Packages not installed in the active virtual environment	Activate it (venv\Scripts\activate), then run pip install -r requirements.txt
PowerShell: “running scripts is disabled”	Windows script execution policy	Run Set-ExecutionPolicy -Scope CurrentUser RemoteSigned once, then activate again
Port 5000 already in use	Another instance of app.py is running	Stop it with Ctrl+C, or change the port in app.run() and webbrowser.open()
QR code does not open on the phone	Different Wi-Fi, or the firewall blocks Python	Use the same Wi-Fi, allow Python through the firewall, open the app at http://127.0.0.1:5000 on the PC
Citizen sees no challans	The vehicle number given at registration does not match the records	Register with the exact plate (e.g. TN09AB1234) or use the demo citizen ravi
Sample data missing, or forgotten password	An older violations.db already exists	Stop the server, delete the instance folder and run python app.py again (this resets all data and demo logins)
Browser does not open, or CSS changes not visible	Browser blocked, or cached page	Open http://127.0.0.1:5000 manually; hard-refresh with Ctrl+F5

Scope of the Project

- Lets traffic officers record violations digitally and mark them Paid or Unpaid.
- Provides quick lookup of a vehicle’s history with filters for date, status and violation type.
- Generates a challan with a dynamic QR code that leads to a public status page.
- Gives vehicle owners a separate citizen side to view and pay only their own challans.
- Restricts each side with role-based access and hashed passwords.
- Can be extended with a real payment gateway, SMS or email reminders, CSV / PDF export and ownership verification of vehicles at registration.
- Can use random tokens instead of challan numbers in public links, and can be deployed on a cloud server with a larger database such as PostgreSQL.

Methodology

1. Analyse the requirements: record violations, payment status, search and filters, QR challan, public status page, officer login and a citizen side.
2. Design the database with a Violation table and a User table with a role column.
3. Set up the Flask project and configure SQLAlchemy with SQLite.
4. Implement authentication with Flask-Login, hashed passwords and role-based decorators.
5. Implement the officer features: add violation, history with search and filters, mark paid, edit and delete.
6. Generate the QR code for each challan and build the public status and See Status pages.
7. Implement the citizen side: registration, My Challans and the payment page.
8. Add the officer account page and the sample data loaded on the first run.
9. Build the Bootstrap templates and test every route, including invalid input and access to the wrong side.

Implementation

The project is implemented in Python with Flask. Flask-SQLAlchemy maps the User and Violation classes to SQLite tables and all create, read, update and delete operations go through SQLAlchemy queries. Flask-Login keeps the session, and two decorators built on it, @officer_required and @citizen_required, decide which side of the system a logged-in user can open. Passwords are hashed with Werkzeug.

When an officer submits the Add Violation form, the input is validated, the vehicle number is normalised and a row with the status Unpaid is saved. The challan page then shows the record with a QR code that encodes the address of the public status route, so anyone who scans it sees the details and the payment status without logging in. A citizen's My Challans page filters the same table by the plates saved on the citizen's account, and the payment page changes the status to Paid. The routes are summarised below.

Route	Method	Access	Purpose
/login, /logout	GET, POST	Public / any user	Sign in (Officer or Citizen tab) and sign out
/register	GET, POST	Public	Citizen account with vehicle number(s)
/add, /history	GET, POST	Officer	Add a violation; search and filter history
/violation/<id>[/edit]	GET, POST	Officer	Challan with QR code; edit a record
/violation/<id>/pay, /delete	POST	Officer	Mark Paid; delete a record
/account	GET, POST	Officer	Change password; add an officer
/my-challans	GET	Citizen	Citizen's own challans and amount due
/pay/<id>	GET, POST	Citizen	Pay one of the citizen's own challans
/qr/<id>.png, /status/<id>	GET	Public	QR image; public details and payment status
/status	GET	Public	See Status lookup by challan and vehicle number

The next table shows how each requirement of the project description is met.

Requirement	How it is implemented
Add violation record; automatically Unpaid	/add route and form; status saved as Unpaid
View history; search by vehicle number	Vehicle filter in /history (partial, case-insensitive)
Filter by date, status or violation type	Date range, status and type filters, which can be combined
Update status from Unpaid to Paid	Mark as paid button (officer); Pay now (citizen)
QR code on the challan, leading to a public status page	/qr/<id>.png in challan.html; /status/<id> needs no login
Secure officer login; only logged-in users add or modify	Flask-Login, hashed passwords, @officer_required
Bootstrap-based responsive UI with simple navigation	Navigation bar: Add Violation, View / Search History, See Status
CRUD using SQLAlchemy / SQLite	Create, read, update (pay, edit) and delete routes
User side: view and pay own challans only	Citizen role, /my-challans and /pay/<id> restricted to own vehicles

Experience

- Gained practical knowledge of Flask routing, templates, sessions and flash messages.
- Learned to design database models and perform CRUD operations with Flask-SQLAlchemy and SQLite.
- Learned to protect routes with Flask-Login and to build role-based access with custom decorators.
- Learned to hash passwords and to make sure users can only reach their own data.
- Learned to generate QR codes dynamically and serve an image directly from memory.
- Gained experience building a responsive Bootstrap interface, a printable challan and a payment flow.
- Gained hands-on experience in validating input, loading sample data and testing an application end to end.

Conclusion

The Violation Register digitises the recording and tracking of traffic violations. Officers can record a violation, search a vehicle's history, filter records and mark fines as paid, and each challan carries a QR code that opens a public page showing the violation details and payment status. Vehicle owners get their own side where they can log in, view only their challans and pay them.

The project covers Flask routing, SQLAlchemy with SQLite, authentication with role-based access, QR code generation and a Bootstrap interface, and it meets all the requirements of the project description. It can be extended with a real payment gateway, notifications, ownership verification, exports and cloud deployment.